DART Filing Readiness

공시 전 문서 검증을 돕는 웹 애플리케이션

회계사의 반복 점검 업무에서 발견한 문제를
AI와 함께 작동하는 웹 앱으로 구현하기

최근일 · 2026505521

삼일회계법인 재직 회계사 · Assurance 업무
작성일: 2026년 10월 28일
제출: LearnUs · 2026년 10월 28일(수) 자정

저의 업무에서 출발한 아이디어

저는 삼일회계법인에서 Assurance 업무를 수행하는 회계사입니다. 기업이 작성한 재무제표를 감사·검토하고, 그 결과가 공시되는 과정에 참여하고 있습니다.

공시 직전에는 재무제표의 회계적 내용뿐 아니라 회사명·보고기간·날짜·서식·첨부 문서가 올바르게 맞물리는지도 확인해야 합니다. 삼일회계법인의 R&Q 부서 공시팀은 이러한 공시문서의 구조와 기재 사항을 검증합니다.

분기마다 제출이 집중되면 공시팀은 다수의 인턴을 채용해 체크리스트를 수기로 확인하지만, 반복 작업의 병목과 오류가 남습니다. 이 앱은 기계적으로 확인할 수 있는 항목을 먼저 점검하고, 사람이 판단할 부분과 원문 근거를 함께 보여주기 위해 만들었습니다.

> **웹 앱**
> https://dart-filing-readiness-keunil.tasty-note-8705.chatgpt.site
> **GitHub**
> https://github.com/Keunil/dart-filing-readiness

다음 장에서 재무제표 공시와 DART·DSD를 먼저 설명한 뒤, 앱의 기능과 AI 협업 과정을 소개합니다.

1. 업무 배경과 발견한 문제

상장회사와 일정 요건을 충족하는 비상장회사 등은 적용되는 제도에 따라 재무제표와 관련 보고서를 공시합니다. 투자자와 이해관계자는 공개된 자료를 통해 기업의 재무상태와 경영성과를 확인합니다.

기업은 재무제표를 작성하고, 회계사는 이를 감사해 감사보고서를 작성합니다. 분·반기에는 해당 보고서와 업무 범위에 맞는 검토 절차가 수행되기도 합니다. 본 과제는 이러한 감사·검토의 전문적 판단 전체를 자동화하려는 것이 아니라, **최종 공시문서의 반복적인 형식·구조 점검**을 돕는 데 초점을 맞췄습니다.

처음 접하는 분을 위한 용어

재무제표	기업의 재무상태와 일정 기간의 경영성과 등을 보여 주는 보고서입니다.
Assurance	감사·검토 등 정보의 신뢰성을 높이는 회계사의 업무 영역입니다. 저는 이 분야에서 일하고 있습니다.
DART	금융감독원이 운영하는 전자공시시스템입니다. 기업이 제출한 재무제표와 감사보고서 등을 누구나 조회할 수 있습니다.
DSD	DART 편집기에서 작성·저장하는 전자문서 파일입니다. 표지, 문단, 표, 날짜, 문서 정보 등이 들어 있으며 이 앱이 점검하는 입력 파일입니다.
R&Q 공시팀	삼일회계법인의 R&Q 부서 내에서 공시문서의 구조·기재 사항을 검증하는 팀입니다.

공시까지의 흐름

기업의 재무제표 작성 → 회계사의 감사·검토 → 공시용 문서 작성 → **공시팀의 구조·기재 검증** → DART 제출·공개

앱은 굵게 표시한 단계의 사전 점검을 보조합니다. DSD는 작성·점검용 원본이며, 실제 접수는 DART 편집기의 제출 절차에 따릅니다.

사람을 늘려도 남는 반복 작업

공시가 몰리는 분기에는 짧은 기간에 많은 문서를 확인해야 합니다. 공시팀은 인턴을 다수 채용해 대응하지만, 문서마다 체크리스트를 전부 수기로 확인하는 방식은 인수인계와 재검토가 반복되고 누락·오기 가능성도 남습니다.

따라서 파일을 읽어 오류 후보를 먼저 찾고, 관련 원문으로 바로 이동하며, 수동 확인이 필요한 항목을 구분하는 도구를 기획했습니다. 처리시간이나 오류 감소율은 측정하지 않았으므로 효과를 수치로 단정하지 않습니다.

업무 흐름과 병목은 저의 실무 경험 및 본 과제의 문제 정의를 바탕으로 작성했습니다. 시스템·파일 형식 참고: 금융감독원 DART, 기업공시 길라잡이(전자문서 작성·제출).

REQUIREMENTS & USER FLOW

2. 주요 기능과 사용 순서

핵심 사용자는 접수 전 보고서의 구조를 확인하는 담당자다. 공개 방문자도 실제 업무 파일을 준비하지 않고 가상 회사의 DSD로 동일한 점검 흐름을 체험할 수 있게 했다.

주요 기능	사용자가 확인할 수 있는 동작
DSD 선택·체험	자신의 파일을 선택하거나 합성 샘플을 실행해 문서 정보를 확인한다.
감사인 기준 지정	이름·CIK·주소·대표자를 설정한다. 기본 프로필은 가상 값이다.
서식별 점검	연차 감사보고서와 분·반기 검토보고서의 규칙을 구분해 적용한다.
상태 구분·검색	오류·확인 필요·수동 확인·통과·해당 없음을 구분하고 필요한 항목을 찾는다.
원문 근거 확인	발견 사항의 헤더 정보와 문단·표 등 원문 위치를 확인한다.
확인 기록·내보내기	사람이 검토한 표시를 자동 판정과 구분하고 결과를 내려받는다.

사용 순서

01 · 시작
가상 체험을 실행하거나 DSD 파일을 선택한다.

02 · 기준 확인
해당 감사인의 프로필과 문서 종류를 확인한다.

03 · 결과 검토
상태별로 걸러 근거와 원문을 확인한다.

04 · 수동 확인
사람의 판단이 필요한 항목을 검토한다.

05 · 비교 체험
오류·수정 샘플을 실행해 결과 차이를 본다.

06 · 결과 보관
필요한 결과를 사용자 기기에 내려받는다.

지원 범위

연차 감사보고서 코드는 `00760` · `00761`, 분·반기 검토보고서 코드는 `90020` · `90220` · `90320` · `90420` 이다. 범위 밖이나 코드 미확인 입력에는 연차 규칙을 임의로 적용하지 않는다.

연차 점검은 기본 체크리스트 55개와 보조 점검으로 구성된다. 항목의 적용 여부와 자동·수동 확인 범위가 다르므로 전체 항목 수를 자동 검증된 항목 수로 해석하지 않는다.

> **완료 기준**
> 공개 URL에서 체험 실행 → 결과 표시 → 원문 근거 확인 → 결과 다운로드를 수행할 수 있고, 수동 확인이 남은 상태를 자동 통과로 표시하지 않아야 한다.

3. 앱 구조와 작동 원리

6주차 강의의 HTML·CSS·JavaScript 역할 구분으로 구조를 설명했다. 9주차의 저장 방식 비교를 바탕으로, 문서 보존이 필요 없는 이 앱에서는 브라우저 메모리 처리를 선택한 이유를 점검했다.

HTML·CSS·JavaScript를 제공하는 정적 웹 앱으로 구성했다. 별도 데이터베이스나 로그인, 외부 AI API 없이 파일 읽기부터 결과 생성까지 브라우저에서 수행한다.

구성	역할
화면 · HTML/CSS	파일 선택, 체험, 프로필 입력, 상태별 결과와 원문 근거를 배치한다.
동작 · JavaScript	사용자 동작을 받아 읽기·점검을 실행하고 필터·확인 표시·다운로드를 처리한다.
DSD 읽기 모듈	DSD는 ZIP 형식이다. 내부의 contents.xml·meta.xml에서 헤더·문단·표와 위치를 읽는다.
점검 엔진	보고서 코드에 맞는 규칙을 실행하고 상태·발견 사항·원문 위치를 반환한다.
합성 체험 자료	가상 회사의 XML을 브라우저에서 ZIP으로 만들어 실제 파일과 같은 경로로 처리한다.
정적 호스팅	앱 소스 파일만 제공한다. 사용자가 고른 DSD를 업로드받는 API는 두지 않는다.

기존 도구의 문서 읽기·점검 모듈 7개를 재사용하고 독립 화면을 구성했다. 별도·연결 문서는 각각 점검할 수 있지만 두 파일의 비교와 전체 공시사례 검색은 이번 앱에 포함하지 않았다. 원본 기준과 변경 내역은 docs/provenance.md에 남겼다.

데이터 처리

처리 대상은 공시 전의 **미공시 DSD**일 수 있다. 원본 DSD와 분석 결과를 브라우저 메모리에 두고 서버로 보내거나 저장하지 않는다. 콘텐츠 보안 정책에 `connect-src 'none'`을 적용해 점검 중 네트워크 연결을 제한한다. 파일과 결과가 localStorage에 저장되는 구조로 설명하지 않는다.

사용자가 결과를 내려받으면 그 파일은 사용자 기기에 남는다. 웹 페이지 새로고침·닫기와 다운로드한 파일의 삭제는 서로 다른 동작이다.

재사용과 이번 과제의 변경

기존에 개발하던 DSD 읽기·점검 기능 일부를 활용했다. 이번 과제에서는 이를 독립 웹 앱으로 구성하고, 감사인 기준 입력·가상 체험·결과 확인·내보내기와 공개 배포를 완성했다.

화면과 보고서는 PwC 디자인 가이드에 맞춰 흰 배경, 오렌지 강조, 기존 공식 로고와 UI 토큰을 활용했다. 사내 메뉴·내부 링크·실제 고객 문서는 포함하지 않았으며, 공개 체험은 가상 자료로 구성했다.

> **설명할 수 있는 작동 원리**
> "서버는 화면 파일만 보냅니다. 브라우저가 DSD의 ZIP/XML을 읽어 문서 종류에 맞는 규칙을 실행하고, 결과와 근거 위치를 표시합니다. 의미 판단이 필요한 부분은 사람이 확인합니다."

4. 강의에서 배운 내용을 어떻게 적용했는가

이번 보완 작업에서는 강의안을 다시 읽고, 앱의 설계와 검증 절차를 수업에서 다룬 개념으로 설명해 보았다. 강의 용어를 나열하기보다 코드와 결과에서 확인할 수 있는 연결을 중심으로 정리했다.

참고 강의	과제에서의 적용과 확인 지점
5주차 프롬프트·콘텍스트 엔지니어링	실무 문제, 과제 안내, 기존 코드, 디자인 기준을 구분해 제공했다. 이번에는 실제 오류 DSD 28건을 추가했다. 목적·자료·수행 조건이 분명해야 AI의 결과도 검토할 수 있었다.
6주차 웹의 기본 구조와 작동 원리	`index.html` 은 파일·프로필 입력과 결과 표, `app.css` 는 배치와 반응형 표현, `app.js` 는 이벤트와 화면 갱신을 담당한다. 웹 서버에서 받은 코드를 브라우저가 실행한다.
7주차 웹 배포 방법	GitHub의 소스 저장과 웹 호스팅의 역할을 구분했다. 강의의 Vercel 예시 대신 기존 Sites 공개 배포를 유지했다. 저장소에 파일이 있다는 사실만으로 접속·실행 확인을 대신하지 않았다.
7주차 하네스 엔지니어링 §2·§3	AGENTS.md, 별도 worktree, .tmp 폴더, 회귀 시험, 검토 기록을 사용했다. 기존 개발 기록의 독립 리뷰와 이번 재실행 결과는 서로 다른 근거로 표시했다.
7주차 하네스의 검증 원칙	실행 성공·비교 완료·품질 승인을 나눴다. 28건 처리와 반복 결과 일치는 확인했지만 담당자 정답과 대조하지 않아 정확도 승인은 남겨 두었다.
9주차 웹 앱에서 DB의 역할 §1·§2·§10	저장 필요를 먼저 판단했다. 이번 앱은 문서 1회 점검이 목적이므로 메모리만 사용한다. localStorage나 DB에 고객 문서를 남기지 않는다. 새로고침하면 결과가 사라지는 한계도 함께 설명한다.

> **수업 내용을 적용하며 달라진 점**
> "화면을 만들어 달라"에서 끝내지 않고, 어떤 자료로 무엇을 확인할지 정했다. 특히 가상 감사인 기준을 실제 파일에 적용한 비교와 기준을 지정한 비교를 분리한 것이 이번 검증에서 가장 중요한 판단이었다.

출처: 주차별 로컬 강의안. 상세 파일명·절·SHA-256은 docs/course-alignment.md에 기록했다. 기존 앱의 모든 설계가 처음부터 해당 강의에서 비롯되었다고 소급해 주장하지 않는다.

5. 시작 화면과 점검 결과

캡처는 실제 고객 문서 대신 공개 가능한 합성 DSD를 사용한 화면이다. 방문자는 시작 화면에서 체험을 실행하고 점검 결과로 이동할 수 있다.

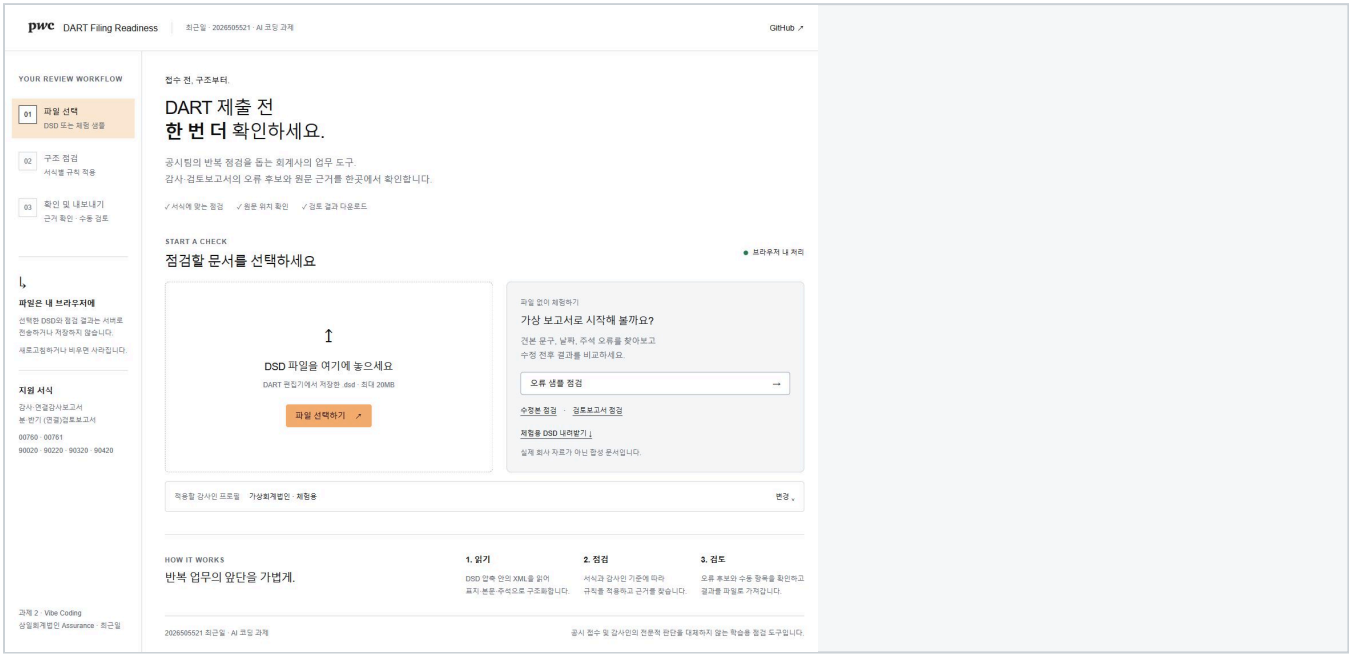


그림 1. 시작 화면. 목적과 처리 방식을 확인하고 가상 체험 또는 파일 선택으로 시작한다.

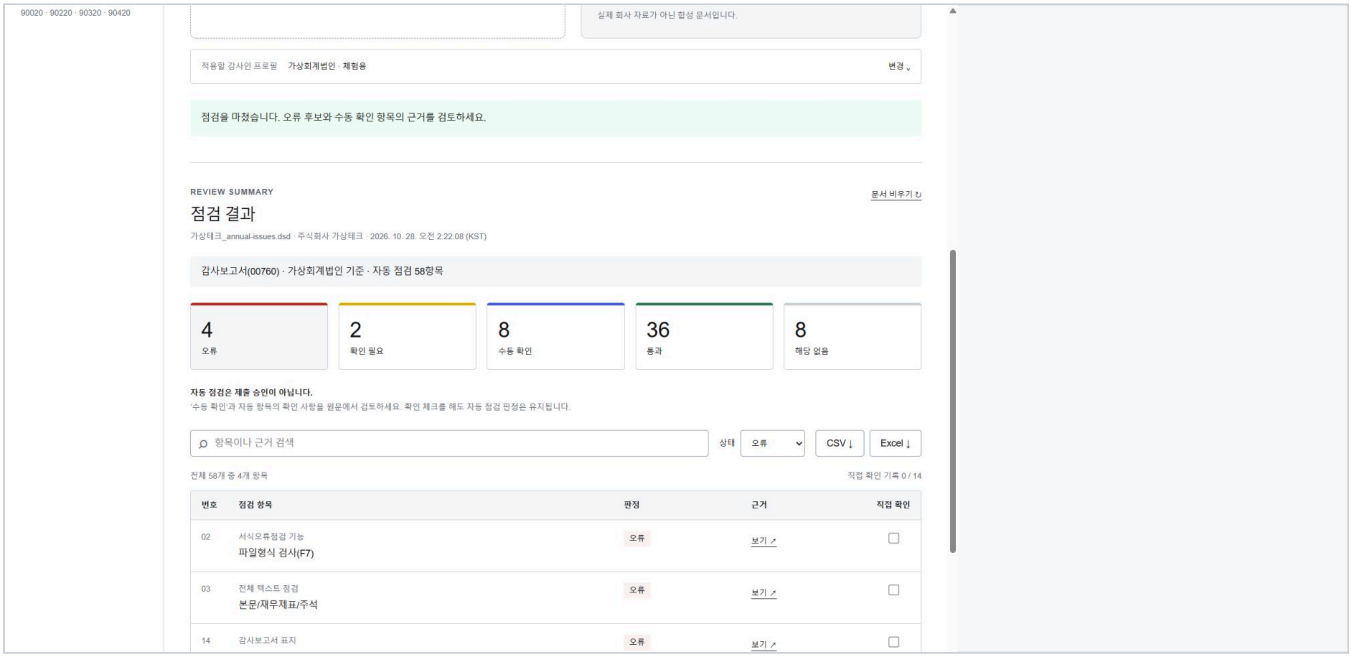


그림 2. 점검 결과. 오류·확인 필요·수동 확인 등의 상태를 구분하고 필요한 결과를 찾는다.

화면 캡처는 해당 시점의 기능 동작을 보여 주는 자료다. 특정 샘플의 결과를 실제 전체 보고서에 대한 정확도나 제출 승인으로 일반화하지 않는다.

6. 원문 근거와 모바일 화면

결과 문구만으로는 무엇을 고쳐야 하는지 알기 어렵다. 발견 사항과 관련 원문 위치를 함께 보여 주어 사용자가 직접 대조하도록 구성했다.



그림 3. 근거 확인. 헤더 정보·본문 위치와 점검 결과를 대조한다.

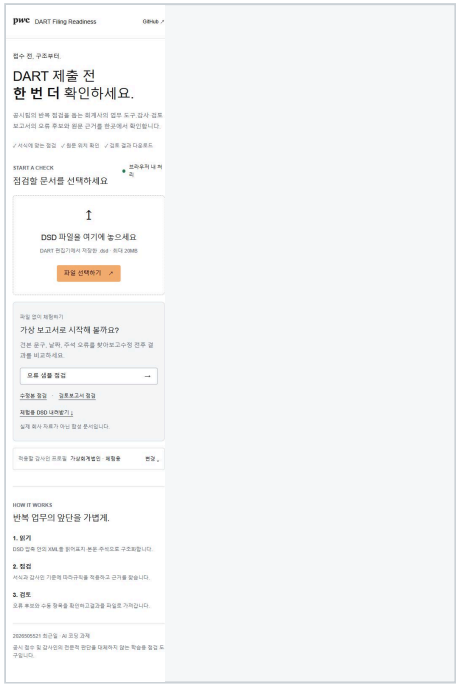


그림 4. 모바일 화면. 좁은 폭에서 핵심 조작과 결과를 확인한다.

판정과 검토 표시

상태	의미
오류	적용한 규칙과 다른 내용이 발견되어 수정·확인이 필요하다.
확인 필요	자동 판정으로 확정하기 어려워 근거를 대조해야 한다.
수동 확인	문서 밖 정보나 사람의 판단이 필요한 부분이 남아 있다.
통과	해당 자동 규칙에서 발견한 문제가 없다는 뜻이다.
해당 없음	문서의 조건에 따라 해당 항목을 적용하지 않는다.

담당자의 확인 체크는 원래 기계 판정과 별도로 표시한다. 체크를 했다는 사실을 자동 검증 성공이나 보고서 전체의 품질 승인으로 바꾸지 않는다.

7. AI와 함께 문제를 해결한 과정

저는 실무에서 겪은 문제, 과제 요구사항과 디자인·설명 방향을 제시했습니다. AI 코딩 도구 Codex에는 기능 구현과 실행·오류 수정·검증을 맡겼습니다. 아래는 기존 개발 기록에 남아 있는 과정입니다. 실제 자료 28건을 추가한 이번 작업은 뒤의 검증 절에 따로 기록했습니다.

A. 특정 감사인의 고정값을 적용하는 문제

문제 상황 원본의 특정 감사인 이름·CIK·주소·대표자 고정값을 독립 앱에 그대로 쓰면 다른 감사인의 보고서도 오류로 판정할 수 있었다.

AI 활용·해결 과정 Codex가 원본의 의존성과 고정 기준을 조사하고, 네 값을 사용자가 입력하는 프로필로 바꾸도록 구현 범위를 정했다. 기본값은 가상회계법인으로 제공한다.

최종 결과·한계 지정 프로필로 확인하는 흐름을 과제용으로 구성했다. 사용자가 입력한 기준의 최신성·정확성을 앱이 공식 인증하는 것은 아니다.

B. 검토보고서에 연차 기준을 적용하는 과탐

문제 상황 연차와 분·반기 문서에 같은 기준을 적용하면 올바른 검토보고서 표현도 오류가 될 수 있다.

AI 활용·해결 과정 최신 원본을 조사한 결과 서식별 분리 구현이 이미 있었다. 이를 재사용하고, 범위 밖·코드 미확인 입력을 임의로 점검하지 않게 구성한다.

최종 결과·한계 기존 보호 동작을 보존하는 재사용 결정이다. 이번 과제에서 새로 발견해 원본 규칙을 전면 수정했다고 쓰지 않는다.

C. 업무 파일이 없는 방문자의 재현 문제

문제 상황 파일 선택만 있으면 평가자가 DSD를 준비하지 못할 때 핵심 기능을 확인할 수 없다.

AI 활용·해결 과정 실제 고객 자료 대신 가상 회사 XML·ZIP을 브라우저에서 생성하는 체험을 구성했다. 수정 전후와 검토보고서를 같은 읽기·점검 경로로 처리한다.

최종 결과·한계 별도 자료 없이 실행할 수 있는 경로를 만들었다. 합성 자료의 기대값 일치는 실제 전체 문서의 정확도 검증과 구분한다.

D. 자동 통과와 수동 확인의 혼동

문제 상황 자동 점검이 끝나도 문장 의미·보고서 범위처럼 사람이 판단할 내용은 남는다.

AI 활용·해결 과정 기존 엔진의 다섯 상태를 화면에서 구분하고, 담당자의 확인 표시를 기계의 원래 상태와 따로 다루도록 요구했다.

최종 결과·한계 원본의 수동 판단 경계를 보존했다. 최종 업무 품질 승인은 실제 담당자의 확인이 필요하다.

위 네 사례는 구조·재사용 결정과 개선 사항이다. 실제 실행 중 발견·수정한 기능 오류와 경계 입력 시험은 다음 절에 따로 기록한다.

8. 실행 중 발견한 오류와 수정

아래 두 사례는 기존 개발 기록에서 구현 후 실행·시험 과정에 확인한 문제다. AI가 앱을 실행하고 관측한 내용이며, 실제 사용자 UAT나 실제 보안 사고로 표현하지 않는다.

E. 근거 창을 열면 닫기 버튼이 사라짐

문제 상황 Agent가 브라우저에서 원문 근거 dialog를 열자, 블록 위치를 찾는 `scrollIntoView()`가 원문 패널의 조상인 dialog까지 스크롤했다. 제목과 닫기 버튼이 화면 위로 밀려나 보이지 않았다.

AI 활용 Codex는 실제 화면에서 제목·닫기 버튼 위치를 확인해 재현했다. 원문 위치로 이동하는 동작을 내부 패널에만 적용하도록 수정 방향을 정했다.

해결 과정 `public/app.js`의 `highlightLocation()`에서 `source-view.scrollTop`만 조절하도록 바꾸었다. dialog를 열 때는 `dialog.scrollTop = 0`으로 초기 위치를 고정했다.

최종 결과 같은 화면을 다시 실행한 시험에서 닫기 버튼 노출(`visible=true`)을 확인했다. 원문은 내부에서 이동하고 바깥 제목·닫기 조작은 유지되었다. 모든 브라우저에서의 동작까지 시험한 것은 아니다.

> **실제 수정 순서**
> AI의 브라우저 실행 → 근거 창의 조상 스크롤 관측 → 내부 패널만 스크롤하도록 수정 → 재실행하여 닫기 버튼 노출 확인

F. 원문이 Excel 수식으로 해석될 수 있는 입력

문제 상황 원문을 CSV로 내보낼 때 글이 `=` · `+` · `-` · `@`로 시작하면 Excel에서 수식으로 해석될 수 있다. 실제 공격 사례가 아니라 내보내기 경계 입력을 시험하며 확인한 조건이다.

AI 활용 Codex가 `=HYPERLINK(...)` 형태의 문자열을 fixture로 구성해 결과 파일이 수식 대신 문자열로 처리되도록 확인하는 시험을 만들었다.

해결 과정 CSV는 선행 공백 뒤의 수식 접두 문자를 검사해 작은따옴표를 붙인다. XLSX는 `inlineStr` 문자열 셀로 기록한다. 코드와 테스트에서 보호 접두 문자 및 문자열 셀 구조를 확인한다.

최종 결과 경계 입력 fixture를 포함한 Node 테스트가 통과했고 브라우저에서 CSV·XLSX 다운로드도 성공했다. 모든 Excel 버전의 실제 열기 동작을 확인한 것은 아니며 실제 피해가 있었다고 주장하지 않는다.

상세 기록: docs/ai-collaboration.md · 재현 및 재실행은 Agent 수행. 사용자의 직접 UAT는 미실행이다.

9. 리뷰와 사용성 개선

G. 비워진 화면에 남은 문서 텍스트

문제 상황 독립 리뷰에서 '문서 비우기'가 결과를 가리기만 하고 결과행·근거·헤더·파일 메타의 텍스트와 이벤트를 남기는 것을 발견했다. 메모리에서만 처리한다는 설명을 지키려면 숨김과 제거를 구별해야 했다.

AI 활용 개발에 사용한 세션과 별도의 Codex 검토 세션에 같은 코드 버전을 읽기 전용으로 확인하도록 맡겼다. 검토자가 실제 초기화 함수를 화면 요소를 재현한 메모리 모형에서 실행해 잔존을 재현했고, 작성 Agent가 수용했다.

해결 과정 공용 초기화 함수에서 상태·원문 뷰·Blob URL과 문서별 화면 요소을 함께 제거했다. 문서 비우기뿐 아니라 새 파일을 읽기 시작하거나 실패하는 경우에도 같은 경로를 사용한다.

최종 결과 수정 전 소스에서 회귀 시험 2개가 실패하고 수정본에서 통과했다. 브라우저에서도 회사명 잔존 false, 근거·헤더·원문·메타 텍스트 길이 0을 확인했다. 브라우저 메모리의 물리적 덮어쓰기를 보증하는 기능은 아니다.

H. 모바일 결과 표의 가로 넘침

문제 상황 좁은 화면에서 결과가 나타나자 문서 폭이 582px로 늘어나 실제 내용 영역 375px를 넘었다. 첫 화면만 봐서는 찾지 못한 문제였다.

AI 활용 Codex가 브라우저의 반응형 폭을 적용해 결과 생성 전후의 페이지 폭을 비교했다. 표의 최소 폭이 grid 자식을 밀어내는 경로를 좁혔다.

해결 과정 main의 최소 폭을 0으로, 표 컨테이너의 최대 폭을 100%로 제한했다. 표의 열은 보존하고 표 안에서만 가로 스크롤하게 했다.

최종 결과 재시험에서 페이지 폭과 문서 폭이 모두 375px로 같았다. 표 내부 334px에 내용 540px만 스크롤된다. 실제 모바일 기기 전체를 시험한 것은 아니다.

10. 실제 오류 사례 28건으로 실행 범위를 넓히기

기존 보고서는 합성 체험 자료 중심이었다. 이번에는 공시팀 안내와 함께 전달받은 실제 DSD 28건을 원본 변경 없이 읽었다. 신규 오류 예시는 만들지 않았고, 실제 파일은 다운로드 폴더에서 직접 읽어 공개본에 복사하지 않았다.

28 / 28건
읽기·점검 완료
실행 예외·범위 제외 0건

23 + 5건
감사보고서 00760
연결감사보고서 00761

1,624항목
28문서 × 58항목
문서 수와 구분하여 집계

자동 점검 상태	문서·항목 조합 수	해석
오류	92	규칙상 수정·확인 대상으로 표시
확인 필요	61	추가 근거 대조 필요
수동 확인	153	사람 판단·문서 밖 자료 필요
통과	888	해당 규칙에서 문제 미발견
해당 없음	430	문서 조건에 따라 비적용

오류가 하나 이상 표시된 문서는 26건이다. 나머지 2건도 보고서 전체가 적정하다는 의미는 아니다. 92는 실제 결함을 확정한 개수가 아니라 **엔진의 오류 판정 항목 수**다. 공시팀 정답표가 없어 정밀도·재현율은 계산하지 않았다.

원본 대신 남긴 재현 근거

`validate-private-dsd.mjs`가 기존 `analyzeDsd()`를 직접 호출한다. 파일별 읽기 성공, 문서 코드, 비어 있지 않은 블록, 판정 개수와 실패한 규칙 번호를 확인한다. 두 번의 연속 실행에서 28건의 입력 해시·판정 집계·실패 규칙이 모두 같았다.

공개 근거에는 DSD-01 같은 익명 ID와 집계만 남겼다. 원문·회사명·파일명·원본 해시는 공개하지 않는다. 기존 가상 체험은 평가자의 접속용으로 유지하되 실제 사례의 대체 정답으로 쓰지 않았다.

> **강의와 연결 — 7주차 하네스**
> 프로그램이 종료됐는지에 더해 28건 모두 점검 단계에 도달했는지를 확인했다. 반복 결과의 일치는 재현성 근거이며, 오류 판정의 정확성을 승인한 근거와는 구분했다.

11. 실제 자료를 넣고 비교 기준부터 다시 확인하다

I. 가상 프로필을 실제 보고서에 적용한 경우

문제 상황 앱의 기본 감사인은 공개 체험용 가상회계법인이다. 이 기준으로 실제 DSD를 점검하면 정상적으로 기재된 감사인 정보도 불일치로 표시된다.

AI 활용 이번 요청에서 실제 오류 자료를 제공하고 코드와 보고서 보완을 맡겼다. Codex는 프로필 의존성을 확인한 뒤 기본값 실행과 지정 프로필 실행을 분리했다. 아래 수치는 이 세션에서 실제로 다시 실행한 결과다.

비교 항목	가상 기본값	지정한 감사인 기준
전체 오류 판정 항목	189	92
4번: 감사인 CIK 오류 문서	28 / 28	0 / 28
14번: 표지 감사인명 오류 문서	28 / 28	1 / 28
확인 필요 / 수동 확인	60 / 153	61 / 153

해결 과정 기존 업무 코드에 있는 삼일회계법인 프로필 스냅숏의 이름·CIK·주소·대표자를 검증 입력으로 지정했다. 프로필은 로컬 .tmp에만 두었다. 비교 결과를 보고 문서 자체나 엔진 규칙을 고치지는 않았다.

최종 결과 이 검증에 필요한 기준을 명시했다. 189에서 92로의 감소를 오류 수정 성과나 정확도 향상으로 쓰지 않는다. 실제 업무에 사용할 때에는 담당자가 기준 정보의 유효성을 확인해야 한다.

규칙별로 다음 검토 대상을 좁히기

규칙	오류 표시 문서 수	다음에 대조할 사항
52 주요 감사실시내용	7	기간·실시내용 원문과 공시팀 지적
18 감사의견	6	회계기준·연결 범위의 문구
27 사업연도 표기	6	재무제표 기간과 표기 일치
2 파일형식 검사	5	DART 편집기 F7 결과와 차이

한 문서가 여러 행에 포함될 수 있다. 위 표는 엔진 출력의 후속 검토 목록이며 개별 오류의 확정 원인을 요약한 것이 아니다. 공시팀 지적 위치가 없는 상태에서 AI가 정답을 추정해 규칙을 맞추지 않았다.

> **강의와 연결 — 콘텍스트 엔지니어링**
> 파일만 제공하는 것으로 충분하지 않았다. "어떤 감사인 기준으로 비교하는가"도 입력 맥락이었다. 결과를 검토하면서 빠진 조건을 찾아 다시 실행한 과정이다.

12. 실행 확인과 비교 결과

검증 결과

개발자 테스트
Node 테스트 11/11 통과, 실패 0, 건너뜀 0. CSV 수식 접두 문자 보호와 XLSX 문자열 셀 fixture를 포함한다.

브라우저에서 확인한 합성 샘플 결과

입력	오류	확인 필요	수동 확인	통과	해당 없음
연차 오류 샘플	4	2	8	36	8
연차 수정본	0	1	9	40	8
검토보고서 샘플	1	11	3	3	1

연차 수정본의 자동 오류는 0개지만 확인 필요 1개와 수동 확인 9개가 남는다. 이를 보고서 전체의 최종 통과나 접수 승인으로 해석하지 않는다. 합성 샘플의 결과 차이는 실제 업무 자료 전체의 정확도·오류 감소 효과와 다르다.

핵심 동작 확인

- 브라우저에서 CSV·XLSX 결과 파일 다운로드 성공.
- 직접 확인 체크 후에도 해당 항목의 자동 오류 판정 유지.
- 근거 창 스크롤 수정 후 닫기 버튼 노출 확인.
- 시작·결과·근거·모바일 주요 화면은 Agent가 캡처.

배포 상태
Sites 버전 2 공개 앱을 Edge 브라우저에서 다시 열어 HTTP 200과 체험 실행을 확인했다. 오류 필터 4행, CSV·XLSX 다운로드, 비우기, 모바일 폭 390px도 재확인했다. 자동 HTTP 클라이언트의 403 응답과 브라우저 접속 결과는 검증 기록에 구분했다.

독립 검토
개발용 AI와 별도의 검토용 AI 세션으로 코드를 확인했다. 화면에 숨겨진 원문이 남는 문제 1건을 재현해 수정하고 재발 방지 시험을 추가했다. 다른 모델 리뷰와 현업 품질 승인은 미실행이다.

미검증과 승인 경계
이번에 실제 DSD 28건의 실행·반복 결과를 확인했다. 사용자의 직접 UAT, 공시팀 정답과의 대조, 오류율·업무시간 감소 효과와 담당자의 품질 승인은 미검증이다. 자동 테스트·Agent 브라우저 확인을 이 단계의 완료로 바꾸지 않는다.

DELIVERABLES & NEXT STEPS

13. 과제 제출물과 남은 단계

이 보고서는 앱 URL·GitHub 주소·개발 목적과 주요 기능·화면 캡처·AI 활용 및 문제 해결 과정을 함께 제시한다. URL 공개, 사용자의 확인, LearnUs 제출은 각각 따로 확인한다.

과제 제출물 대응

필수 제출물	보고서·저장소 위치
웹 앱 URL	https://dart-filing-readiness-keunil.tasty-note-8705.chatgpt.site
GitHub URL	github.com/Keunil/dart-filing-readiness
개발 목적·주요 기능	보고서 1~2절
주요 화면 캡처	보고서 5~6절 · screenshots/
AI 활용·문제 해결	보고서 7~9절·11절 · docs/ai-collaboration.md

평가 기준과 연결

실제 작동(30%)은 공개 체험과 실행 확인, 구조·기능 적절성(20%)은 브라우저 처리와 서식별 점검으로 제시한다. AI 협업(20%)과 문제 해결(20%)은 재사용 결정·실제 화면 오류 수정·경계 입력 시험 기록으로, 배포·제출 완성도(10%)는 URL·GitHub·캡처·보고서로 제시한다.

이번 작업에서 배운 점

회계 업무를 아는 것과 검증 기준을 코드로 전달하는 것은 별개의 일이었다. 실제 문서를 넣으니 체험용 기본값이 결과에 영향을 주었다. AI가 낸 숫자를 곧바로 성과로 쓰기보다, 입력 조건과 사람이 확인할 범위를 먼저 정하는 것이 제 역할이라고 느꼈다.

확장 방향

이번에 읽은 28건을 담당자의 정답과 대조해 과탐·누락을 확인하고, 서식 변화에 맞춰 규칙과 회귀 시험을 갱신할 수 있다. 이후 실제 담당자의 검토시간과 수정 내역을 수집하면 업무 개선 효과를 측정할 수 있다. 현재 보고서는 그러한 효과가 입증되었다고 주장하지 않는다.

> **제출 전 확인**
> 공개 앱과 GitHub 링크를 직접 열어 체험을 확인한 뒤 개발 보고서를 LearnUs에 제출한다. 보고서 생성과 LearnUs 제출은 별도 단계다.